

# Review Report: Add Bill Option Under Payment View

|                                               |                                                                                     |
|-----------------------------------------------|-------------------------------------------------------------------------------------|
| Application: Recordkeeper                     | Area Reviewed: Add File / Payment view, Add Bill option, reports and search linkage |
| Prepared For: Printing and operational review | Prepared On: 01 September 2026                                                      |

## 1. Executive Summary

The new Add Bill option is an explicit bill-entry facility inside the Payment view. It allows a user to create one or more bill records for a supply order, record bill amount, bill preparation date, bill sent date, return/resubmission cycles, payment date, payment mode, actual payment amount, and remarks. The option is functionally connected to cash-outgo reporting, bill submission reporting, actual payment reporting, returned-bill reports, Cash Out Go Plan calculations, and payment pending/completed status filters. Internally, each added bill is converted into an effective payment-stage row so that report builders can treat it like a bill-level financial event.

**Overall assessment:** The feature is useful and mostly integrated, but it is not yet a full bill ledger as described in the implementation note. The present build stores bill entries inside a JSON field on the supply order. This works, but some behavior should be corrected or clarified before relying on it as the final audited bill-management model.

### Primary Strengths

- Supports multiple bill entries against one supply order.
- Supports normal, combined stage, supplementary, and advance bill types.
- Connects bill preparation, bill sent, return, resubmission, and payment dates to reports.
- Prevents double counting in several search/status paths by suppressing normal payment rows when explicit Add Bill rows exist.
- Allows stage-wise amount breakup where required.

### Main Corrections Recommended

1. Add chronology validation for Add Bill dates and return cycles.
2. Clarify or enforce exclusive use of either legacy payment fields or Add Bill rows for a supply order.
3. Improve partial-payment handling consistently across SQL reports and utility report builders.
4. Show bill amount as the amount source in Cash Out Go Plan for explicit Add Bill rows.
5. Link supplementary and previous-unpaid options to actual balance logic, not only free text/manual reference.

## 2. How the Add Bill Option Works

### Where the Option Appears

The Add Bill option appears under the Payment view of the Add/Edit File workflow. It is implemented in the frontend component `PaymentBillsBlock` in `src/routes/add.tsx`. The block is rendered only when the active subview is Payment.

### Data Captured Per Bill

| Field / Option                      | Purpose                                                                                                                             |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Bill Type                           | Normal, Combined stage, Supplementary, or Advance.                                                                                  |
| Amount Entry Mode                   | Combined total or stage-wise detail.                                                                                                |
| Bill Reference                      | Identifies the bill in the UI and report rows.                                                                                      |
| Supplementary Reason                | Reason such as previous unpaid balance, GST unpaid, rate difference, penalty recovery/refund, balance payment, or other adjustment. |
| Previous Unpaid / Previous Bill Ref | Records whether the bill relates to an earlier unpaid bill and captures a reference.                                                |
| Stage Selection                     | Links the bill to one or more supply-order stages where stage data exists.                                                          |
| Bill and Actual Amounts             | Captures capital and revenue portions separately.                                                                                   |
| Bill Dates                          | Preparation date, sent-for-payment date, returned/resubmitted cycles, payment date.                                                 |
| Payment Mode and Remarks            | Records mode of payment and any operational note.                                                                                   |

### Amount Modes

In Combined total mode, the user enters capital and/or revenue bill amount and actual payment amount directly on the bill. In Stage-wise detail mode, the user selects stages and enters line-wise values for each selected stage. If direct bill amounts are blank, report utilities can derive the bill amount from the line totals.

### Return and Resubmission

Each Add Bill row can carry bill return cycles through the shared returned-bill block. A cycle records return and resubmission information and is then consumed by returned-bill reporting and milestone logic.

### Save Behavior

Add Bill rows are saved as part of the parent Payment view save/update action. There is no separate bill-level save button. Empty placeholder bill rows are filtered out before saving; rows with real reference, amount, date, return cycle, stage selection, payment mode, or remarks are retained.

### 3. Storage and Report Linkage

#### Storage Model

Add Bill data is stored in the `supply_orders.payment_bills` JSONB column. The migration file is `database/069_supply_order_payment_bills.sql`. The frontend and backend share the same bill shape, including bill type, amount mode, stage indexes, previous unpaid flags, supplementary reason, dates, return cycles, payment details, lines, and remarks.

#### Effective Payment Rows

Report code expands each saved payment bill into an effective payment row. This is done by `getPaymentBillOrderEntries` in both frontend and backend effective-delivery utilities. These generated rows carry `paymentBillEntry = "Yes"` and map Add Bill fields into the normal reporting structure.

| Report / Area                                | How Add Bill Links                                                                                                                                                          |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Items Delivered and Bills Yet to Be Prepared | Add Bill rows without bill preparation date can appear as bill-preparation pending items, depending on report mode and report builder path.                                 |
| Bills Submitted to PCDA                      | Rows with bill sent-for-payment date and no payment date can appear as submitted/pending-payment rows.                                                                      |
| Actual Cash Outgo / Spent Till Date FY       | Rows with payment date contribute to actual cash-outgo rows using actual capital and revenue amounts.                                                                       |
| Returned Bill Reports                        | Return cycles under an Add Bill row feed returned, pending returned, resubmitted, and returned-and-paid report groups.                                                      |
| Cash Out Go Plan                             | Add Bill entries are included as explicit payment entries. Their prep, sent, return, and payment dates affect plan classification and expected outgo timing.                |
| Payment Pending / Completed Clickers         | Search and status SQL include explicit Add Bill branches. When Add Bill rows exist, some normal supply-order payment branches are suppressed to prevent duplicate counting. |

#### Milestone Interpretation

If payment date exists, the bill is treated as Payment. If sent date exists and payment is blank, it is treated as Bill Sent for Payment. If preparation date exists but sent date is blank, it is treated as Bill Preparation. If an open return cycle exists, returned-bill status is used.

#### Important Technical References

- `src/routes/add.tsx`: Add Bill user interface and cleaning before save.
- `src/lib/files-store.ts` and `backend/src/types.ts`: shared data structure.
- `src/lib/effective-deliveries.ts` and `backend/src/utls/effective-deliveries.ts`: conversion into report rows.
- `backend/src/routes/reports.ts` and `backend/src/utls/report-summary.ts`: report integration.
- `backend/src/routes/files.ts`: payment pending/completed search and milestone filters.

## 4. Inconsistencies and Risk Areas

- 1. Design document and implementation differ.** The design note describes a fuller bill-payment ledger with dedicated bill, payment, and return-cycle rows. The current implementation stores bills as JSON under the supply order. This is workable, but it provides fewer database-level constraints and makes auditing/report consistency harder.
- 2. Bill type naming differs.** The design note uses language such as regular bills and separate adjustment/GST balance concepts. The current UI uses Normal, Combined stage, Supplementary, and Advance. Supplementary reasons are stored as reason text, not separate report categories.
- 3. Previous unpaid bill is mostly manual.** The UI captures whether a bill includes previous unpaid balance and allows a previous bill reference, but no confirmed automatic calculation or linkage to the earlier unpaid bill balance was found.
- 4. Partial payment handling may diverge between report paths.** SQL report routes include balance-oriented expressions for explicit bill rows in some places. The utility summary path may still use planned/bill amount more broadly. This can create different totals if two report paths are used for the same scenario.
- 5. Add Bill rows may display misleading amount source in Cash Out Go Plan.** Because the generated effective row uses a stage delivery label, Cash Out Go Plan may classify the source as a stage amount instead of bill amount for explicit Add Bill rows.
- 6. Date chronology validation appears incomplete for Add Bill rows.** The app has chronology warnings for normal supply-order, stage, and advance-payment fields, but an equivalent Add Bill-specific warning set was not found. The system should warn if bill sent date is before preparation date, payment date is before sent date, return date is before sent date, or resubmission date is before return date.
- 7. Mixed legacy payment fields and Add Bill rows can be ambiguous.** Some search/status branches suppress normal payment rows when Add Bill rows exist. That prevents double counting, but if a supply order mixes old payment fields and new bill rows, valid legacy payment data may be hidden from some views.
- 8. Stage-wise mode does not fully match the design note.** The design note expects stage amount suggestions and balance checks. The current UI supports stage-wise manual entry, but the review did not confirm automatic stage amount suggestion or warnings when billed totals exceed remaining supply-order value.

## 5. Recommended Corrections and Test Checklist

### High Priority Corrections

1. Add Add Bill chronology warnings: preparation date should not be after sent date; sent date should not be after payment date; return and resubmission cycles should follow correct order.
2. Make partial-payment calculation consistent across SQL report routes and utility summary builders. The same bill should produce the same pending and actual amount in every report.
3. Decide whether a supply order can mix legacy payment fields and Add Bill rows. If not, show a clear UI warning or migrate existing payment fields into Add Bill rows.
4. In Cash Out Go Plan, classify explicit Add Bill rows as bill amount source, not stage amount source.

### Medium Priority Improvements

1. Convert previous unpaid bill reference into an actual selectable/linkable bill with automatic unpaid balance pickup.
2. Add stage amount suggestions and overbilling warnings in stage-wise mode.
3. Give supplementary bill categories a reportable code, not only a display reason.
4. Allow report/search results to drill down directly to the relevant bill row inside the Payment view.

### Functional Test Checklist

| Scenario                                     | Expected Result                                                                                  |
|----------------------------------------------|--------------------------------------------------------------------------------------------------|
| Add normal bill with prep date only          | Appears as bill prepared / pending sent where applicable.                                        |
| Add bill with sent date but no payment       | Appears under Bills Submitted to PCDA / payment pending.                                         |
| Add bill with payment date and actual amount | Appears under actual cash outgo with capital/revenue split.                                      |
| Add returned bill cycle without resubmission | Appears under pending returned bills and current returned status.                                |
| Add returned bill cycle with resubmission    | Moves into resubmitted returned-bill reporting.                                                  |
| Add stage-wise combined bill                 | Line totals should equal report amount and should not double-count stages.                       |
| Add supplementary previous unpaid bill       | Should either import unpaid balance automatically or clearly remain manual.                      |
| Add advance bill                             | Should be marked as advance in cash-outgo reporting and avoid duplicate advance-payment entries. |

**Conclusion:** The Add Bill option is connected to the major cash-outgo and payment reports, and it is suitable for structured bill capture. Before it becomes the single authoritative bill ledger, the main work is to strengthen validation, balance logic, previous-unpaid linkage, and consistency between report paths.